CHAPTER 6

Detecting periodicities with Fourier analysis

Sinusoidal oscillations—those involving sines and cosines—are very common in the environmental sciences. We encountered them in the Neuse River Hydrograph and the Black Rock Forest datasets, where they were associated with seasonal variations in river discharge and air temperature, respectively. This chapter examines oscillatory behavior in more detail, developing systematic methods for detecting and quantifying periodicities.

6.1 Describing sinusoidal oscillations

Periodicities can be both temporal and spatial in character, with a somewhat different nomenclature used for each. In both cases, the height of the oscillation is called its *amplitude*. Temporal periodicities have a *period*, T, the time between successive cycles. Spatial periodicities have a *wavelength*, λ, the distance between successive cycles. The rate at which temporal cycles occur is called the *frequency,* and the rate at which spatial cycles occur is called *wavenumber*. Frequency can be measured in units of cycles per unit time, in which case it is given the symbol, f, or it can be measured in units of radians per unit time, in which case it is called *angular frequency* and given the symbol, ω. The units of cycles per second are called Hertz, abbreviated Hz.

Similarly, wavenumber can be measured in either cycles per unit distance or radians per unit distance. Unfortunately, both units of wavenumber tend to be given the same symbol, k, in the literature. In this book, we use k exclusively to mean radians per unit distance. These quantities are related as follows:

$$f = \frac{1}{T} = \frac{\omega}{2\pi} \quad \text{and} \quad \frac{1}{\lambda} = \frac{k}{2\pi} \tag{6.1}$$

Generic temporal, $d(t)$, and spatial, $d(x)$, cosine oscillations of amplitude, C, can be written as

$$\begin{aligned} d(t) &= C\cos\left\{\frac{2\pi}{T}t\right\} = C\cos\left(2\pi f t\right) = C\cos\left(\omega t\right) \text{ and} \\ d(x) &= C\cos\left\{\frac{2\pi}{\lambda}x\right\} = C\cos\left(kx\right) \end{aligned} \tag{6.2}$$

Environmental Data Analysis with MATLAB® or Python
https://doi.org/10.1016/B978-0-323-95576-8.00008-8

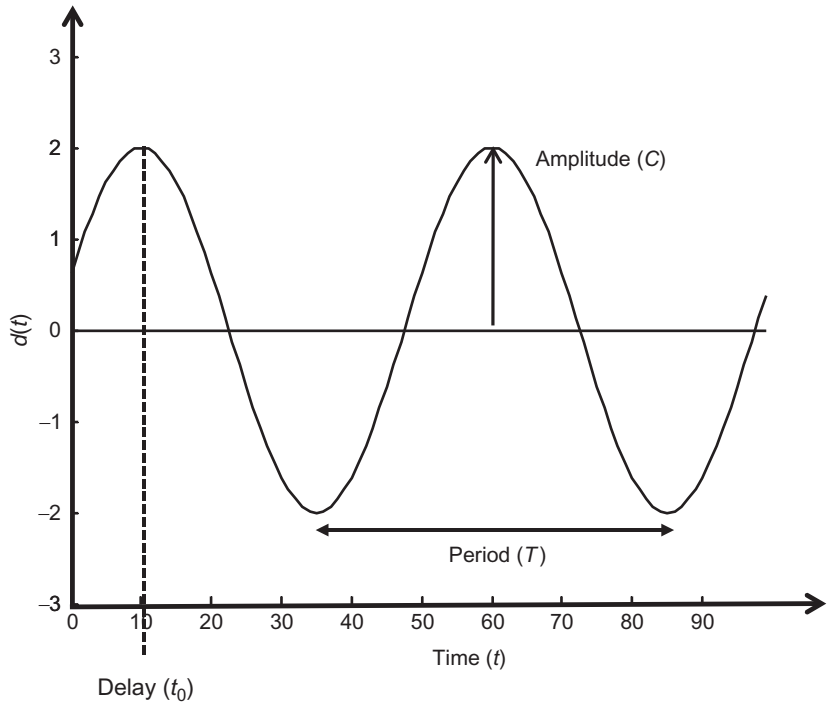


Fig. 6.1 The sinusoidal function, $d(t) = C\cos\{2\pi(t - t_0)/T\}$, has amplitude, $C = 2$, period, $T = 50$, and delay, $t_0 = 10$. Scripts `edama_06_01` and `edapy_06_01`.

In nature, oscillations rarely "start" at time (or distance) zero. A cosine wave with amplitude, C, that starts (peaks) at time, t_0, is given by (Fig. 6.1)

$$d(t) = C\ \cos\left\{\frac{2\pi}{T}(t - t_0)\right\} = C\ \cos\{\omega(t - t_0)\} = C\ \cos\{\omega t - \varphi\} \tag{6.3}$$

The quantity, $\varphi = \omega t_0$, is called the *phase*. The rule,

$$\cos(a - b) = \cos(a)\cos(b) + \sin(a)\sin(b) \tag{6.4}$$

when applied to Eq. (6.3), yields

$$\begin{aligned} d(t) &= C\cos\{\omega t - \varphi\} \\ &= C\cos\{\omega t_0\}\cos\{\omega t\} + C\sin\{\omega t_0\}\sin\{\omega t\} \\ &= A\cos\{\omega t\} + B\sin\{\omega t\} \end{aligned}$$

with (6.5)

$$A = C\cos\{\omega t_0\} \text{ and } B = C\sin\{\omega t_0\}$$

and

$$C = \sqrt{A^2 + B^2} \text{ and } t_0 = \omega^{-1}\tan^{-1}(B/A)$$

See Note 6.1 for a discussion of how the arc-tangent is to be correctly computed. A time-shifted cosine can be represented as the sum of a sine and a cosine. An explicit time-shift variable such as t_0 is unnecessary in formulas describing periodicities, as long as sines and cosines are paired up with one another.

6.2 Models composed only of sinusoidal functions

Suppose that we have a dataset in which a variable, such as temperature, is sampled at evenly spaced intervals of time, t_i (a time series), say with sampling interval, Δt. One extreme is a dataset composed of only sinusoidal oscillations:

$$\begin{gathered}\text{the data} = \text{sum of sines and cosines} \\ d(t_i) = A_1 \cos\{\omega_1 t_i\} + B_1 \sin\{\omega_1 t_i\} + A_2 \cos\{\omega_2 t_i\} + B_2 \sin\{\omega_2 t_i\} + \cdots\end{gathered} \tag{6.6}$$

This formula is sometimes referred to as a *Fourier series* or an *inverse discrete Fourier transform* and the column-vector containing the As and Bs is called the *discrete Fourier transform (D.F.T.)* of d. The As and Bs are the model parameters and the corresponding frequencies are taken to be auxiliary variables. Note that sines and cosines of a given frequency, ω_j, are paired, as was discussed in the previous section. Two key questions involve the number of frequencies that ought to be included in this representation and what their values should be. The answers to these questions involve a surprising fact about time series:

$$\text{frequencies higher than } f_{ny} = \frac{1}{2\Delta t} \quad \text{cannot be detected} \tag{6.7}$$

This is called *Nyquist's Sampling Theorem*. It says that the periods shorter than two time increments cannot be detected in time series with evenly spaced samples. Furthermore, as we will see below, any frequencies in the data that are higher than the Nyquist frequency, f_{ny}, are erroneously mapped into the $(0, f_{ny})$ frequency interval, an effect called *aliasing*. Choosing the frequency interval $(0, f_{ny})$ in Eq. (6.6) is, therefore, natural. For simplicity, we assume that the number, N, of data is an even integer. It can always be made so by shortening an odd-length time series by one point. Then the number of frequencies is $N/2+1$ and the number M of unknown coefficients of cosines and sines exactly equals the number N of data; that is, $M=N$. These choices imply

$$f_{ny} = \frac{1}{2\Delta t} \quad \text{and} \quad \omega_{ny} = \frac{\pi}{\Delta t} \quad \text{and} \quad \Delta\omega = \frac{2\pi}{N\Delta t} \quad \text{and} \quad \Delta f = \frac{1}{N\Delta t} \tag{6.8}$$

The reason that this relationship calls for $(N/2)+1$, as contrasted to $N/2$ of them, is that the first and last sine term is identically zero; that is,

$$\begin{aligned} B_1 \sin(0\, t_i) = 0 \quad \text{and} \quad B_{(N/2)+1} \sin\left(\frac{\pi}{\Delta t} t_i\right) &= B_{(N/2)+1} \sin\left(\frac{t_i}{\Delta t}\pi\right) \\ = B_{(N/2)+1} \sin(n\pi) &= 0 \end{aligned} \tag{6.9}$$

where $n = t_i/\Delta t$ is an integer. Hence, these two terms are omitted from the sum. Thus, the Fourier series contains the frequencies $[0, \Delta f, 2\Delta f, \cdots, \frac{1}{2}N\Delta f]^T$. The lowest frequency is zero. It corresponds to a cosine of zero period; that is, to a constant. The next highest frequency is Δf. It corresponds to a sine and a cosine that have one complete oscillation over the length of the data. The next highest frequency is $2\Delta f$. It corresponds to a sine and a cosine that have two complete oscillations over the length of the data. The highest frequency is $f_{ny} = \frac{1}{2}N\Delta f = 1/(2\Delta t)$. It corresponds to a highly oscillatory cosine that reverses sign at every sample; that is, $[1, -1, 1, -1, \cdots]^T$ (Fig. 6.2).

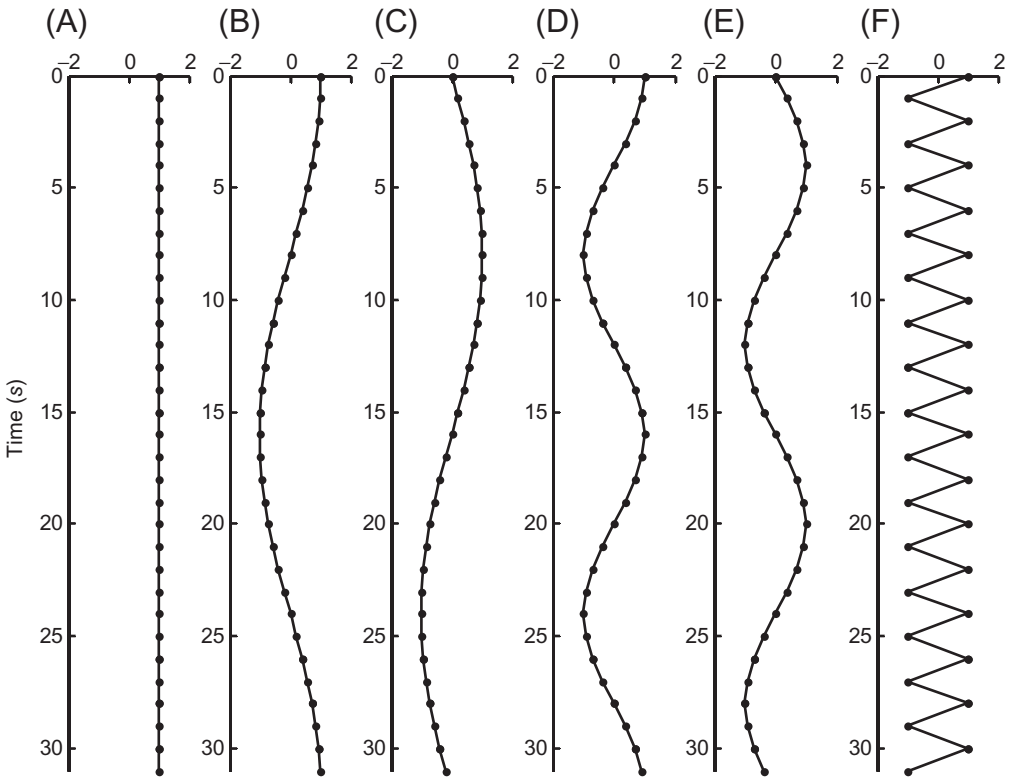


Fig. 6.2 Plots of columns of the matrix, **G**. with rows indicated by solid circles. (A) First column, the constant 1. (B) and (C) Next two columns are $\cos(\Delta\omega t)$ and $\sin(\Delta\omega t)$, respectively. They have one period of oscillation over the time interval of the data. (D) and (E) Next two columns are $\cos(2\Delta\omega t)$ and $\sin(2\Delta\omega t)$, respectively. They have two periods of oscillation over the time interval of the data. (F) Last column switches between 1 and −1 every row. Scripts `edama_06_02` and `edapy_06_02`.

In scripts, frequency-related quantities are defined as follows:

MATLAB®

```
% edama_06_03: example of aliasing
. . .
% standard set-up of Fourier parameters
% note: N required to be an even integer
Nf=N/2+1;  % number of non-negative frequencies
fmax = 1/(2*Dt);  % Nyquist frequency
Df = fmax/(N/2);  % frequency increment
f = Df*[0:Nf-1]';  % non-negative frequency axis
Nw=Nf;             % number of angular frequencies
wmax = 2*pi*fmax; % Nyquist angular frequency
Dw = wmax/(N/2);  % angular frequency increment
w = Dw*[0:Nw-1]';  % non-negative angular frequency axis
```

Python

```
# edapy_06_03: example of aliasing
. . .
# standard set-up of Fourier parameters
# note: N required to be an even integer
Nf=floor(N/2+1);  # number of non-negative frequencies
fmax = 1/(2*Dt1); # Nyquist frequency
Df = fmax/(N/2);  # frequency increment
# non-negative frequency axis
f = eda_cvec( np.linspace(0,fmax,Nf) );
Nw=Nf;             # number of angular frequencies
wmax = 2*pi*fmax; # Nyquist angular frequency
Dw = wmax/(N/2);  # angular frequency increment
# non-negative angular frequency axis
w = eda_cvec( np.linspace(0,wmax,Nw) );
```

In both scripts, `Dt` is the sampling interval and `N` is the length of the data (which is assumed to be an even integer).

The problem that arises with frequencies higher than the Nyquist frequency can be seen by examining a pair of cosines and sines from Eq. (6.6), written out for a particular time, t_k, and frequency, ω_n.

$$\begin{aligned}\cos(\omega_n t_k) &= \cos((n-1)(k-1)\Delta\omega\Delta t) = \cos\left(\frac{2\pi(n-1)(k-1)}{N}\right)\\ \sin(\omega_n t_k) &= \sin((n-1)(k-1)\Delta\omega\Delta t) = \sin\left(\frac{2\pi(n-1)(k-1)}{N}\right)\end{aligned} \tag{6.10}$$

Here, we have used the rule, $\Delta\omega\Delta t = 2\pi/N$. Note that we index time and frequency so that t_1 and ω_1 correspond to zero time and frequency, respectively; that is, $t_k = (k-1)\Delta t$ and $\omega_n = (n-1)\Delta\omega$. Now let us examine a frequency, ω_m, that is higher than the Nyquist frequency, say $m = n+N$, where N is the number of data:

$$\begin{aligned}&\cos(\omega_m t_k) = \cos((n-1+N)(k-1)\Delta\omega\Delta t) = \cos\left(\frac{2\pi(n-1+N)(k-1)}{N}\right)\\ &= \cos\left(\frac{2\pi(n-1)(k-1)}{N}\right)\sin(2\pi(k-1)) - \sin\left(\frac{2\pi(n-1)(k-1)}{N}\right)\sin(2\pi(k-1))\\ &= \cos\left(\frac{2\pi(n-1)(k-1)}{N}\right) - 0\\ &\sin(\omega_m t_k) = \sin((n-1+N)(k-1)\Delta\omega\Delta t) = \sin\left(\frac{2\pi(n-1+N)(k-1)}{N}\right)\\ &= \sin\left(\frac{2\pi(n-1)(k-1)}{N}\right)\cos(2\pi(k-1)) + \cos\left(\frac{2\pi(n-1)(k-1)}{N}\right)\sin(2\pi(k-1))\\ &= \sin\left(\frac{2\pi(n-1)(k-1)}{N}\right) + 0\end{aligned} \tag{6.11}$$

Here, we have used the rules, $\cos(a+b) = \cos a\cos b - \sin a\sin b$ and $\sin(a+b) = \sin a\cos b + \cos a\sin b$, together with $\cos(2\pi(k-1)) = 1$ and $\sin(2\pi(k-1)) = 0$ for any integer, k. Comparing Eqs. (6.10), (6.11), we conclude:

$$\cos(\omega_m t_k) = \cos(\omega_n t_k) \quad \text{and} \quad \sin(\omega_m t_k) = \sin(\omega_n t_k) \tag{6.12}$$

The frequencies, ω_{n+N} and ω_n, are *equivalent*, in the sense that they yield exactly the same sinusoids (Fig. 6.3). A similar calculation, omitted here, shows that ω_{n-N} and $-\omega_n$ are equivalent in this same sense. But sines and cosines with negative frequencies have the same shape, up to a sign, as those with corresponding positive frequencies; that is, $\cos(-\omega t) = \cos(\omega t)$ and $\sin(-\omega t) = -\sin(\omega t)$. Thus, only sinusoids in the frequencies in the range ω_1 (zero frequency) to $\omega_{N/2+1}$ (the Nyquist frequency) have unique shapes (Fig. 6.3C). In a digital world, no frequencies are higher than the Nyquist.

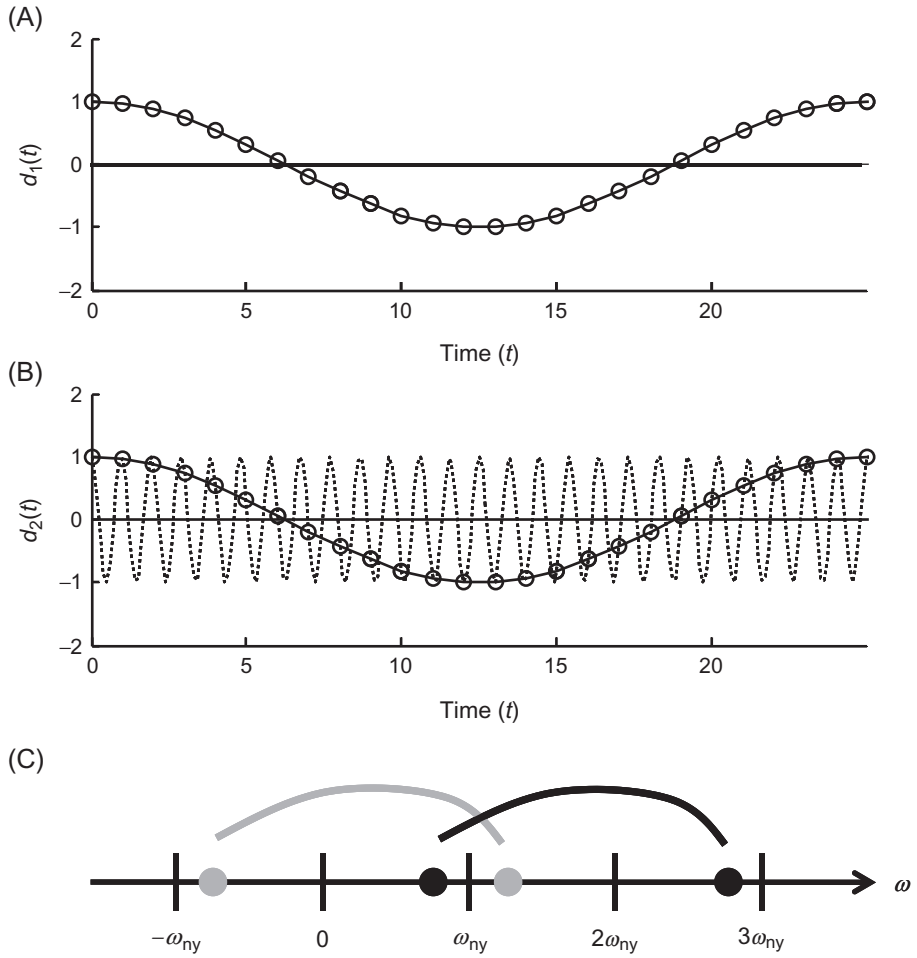


Fig. 6.3 Example of aliasing. (A) Low-frequency oscillation, $d_1(t) = \cos(\omega_1 t)$, with $\omega_1 = 2\,\Delta\,\omega$, evaluated every Δt *(circles)*. (B) High-frequency oscillation, $d_2(t) = \cos(\omega_2 t)$, with $\omega_2 = (N+2)\Delta\,\omega$, evaluated every ωt *(circles)*. Note that both the true curve *(dashed)* and low-frequency curve *(solid)* pass through the data points. (C) Schematic representation of aliasing, showing pairs of points on the frequency-axis that are equivalent. Scripts `edama_06_03` and `04`, `edapy_06_03` and `04`.

This limitation implies that special care must be taken when observing a time series to avoid recording any frequencies that are higher than the Nyquist frequency. This goal is usually achieved by creating a data recording system that attenuates—or eliminates—high frequencies *before* the data are converted into digital samples. If any high frequencies persist, then the dataset is said to be aliased, and spurious low frequencies will appear.

The linear equation, $\mathbf{d}=\mathbf{Gm}$, form of the Fourier series is as follows:

$$\text{the data} = \text{sum of cosines and sines}$$

$$\begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_N \end{bmatrix} = \begin{bmatrix} 1 & \cos(\omega_2 t_1) & \sin(\omega_2 t_1) & \cdots & \cos\left(\omega_{N/2} t_1\right) & \sin\left(\omega_{N/2} t_1\right) & 1 \\ 1 & \cos(\omega_2 t_2) & \sin(\omega_2 t_2) & \cdots & \cos\left(\omega_{N/2} t_2\right) & \sin\left(\omega_{N/2} t_2\right) & -1 \\ 1 & \cos(\omega_2 t_3) & \sin(\omega_2 t_3) & \cdots & \cos\left(\omega_{N/2} t_3\right) & \sin\left(\omega_{N/2} t_3\right) & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 1 & \cos(\omega_2 t_N) & \sin(\omega_2 t_N) & \cdots & \cos\left(\omega_{N/2} t_N\right) & \sin\left(\omega_{N/2} t_N\right) & -1 \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \\ B_2 \\ \vdots \\ A_{N/2} \\ B_{N/2} \\ A_{N/2+1} \end{bmatrix} \tag{6.13}$$

In scripts, the data kernel **G** is constructed as follows:

MATLAB

```
% eda06_05: a.s.d. of the Neuse River hydrograph
. . .
% set up data kernel G
G=zeros(N,M);     % initialize to zero
G(:,1)=ones(N,1); % zero frequency column
% interior M/2-1 columns
for i = [1:M/2-1] % loop over frequencies
    j = 2*i; % index of cosine column
    k = j+1; % index of sine column
    G(:,j)=cos(w(i+1).*t); % evaluate cosine
    G(:,k)=sin(w(i+1).*t); % evaluate sine
end
G(:,M)=cos(w(Nw).*t); % nyquist column
```

Python

```
# edapy_06_05: a.s.d. of the Neuse River hydrograph
. . .
# set up data kernel G
M=N;
Mo2=floor(M/2);
G = np.zeros((N,M));
G[0:N,0:1] = np.ones((N,1)); # zero frequency column
# interior M/2-1 columns.
```

```
for i in range(1,Mo2):
    j = 2*i-1;
    k = j+1;
    G[0:N,j:j+1]=np.cos(w[i,0]*t);
    G[0:N,k:k+1]=np.sin(w[i,0]*t);
G[0:N,M-1:M] = np.cos(w[Nw-1,0]*t); # nyquist column
```

Here, the number of model parameters, `M`, equals the number of data, `N`, and the angular frequency values are in a column-vector, `w`.

Remarkably, the Gram matrix and its inverse, $[\mathbf{G}^T\mathbf{G}]$ and $[\mathbf{G}^T\mathbf{G}]^{-1}$, which play important roles in the least-squares solution, can be shown to be diagonal:

$$\begin{aligned} \text{diag}\left[\mathbf{G}^T\mathbf{G}\right] &= N[1, \tfrac{1}{2}, \tfrac{1}{2}, \cdots \tfrac{1}{2}, \tfrac{1}{2}, , 1]^T \\ \text{diag}\left[\mathbf{G}^T\mathbf{G}\right]^{-1} &= \frac{1}{N}[1, 2, 2, \cdots 2, 2, , 1]^T \end{aligned} \tag{6.14}$$

Note that we have used the rule that the inverse of a diagonal matrix, $\mathbf{M}$, is the matrix of reciprocals of the elements of the main diagonal; that is $[\mathbf{M}^{-1}]_{ii} = 1/M_{ii}$. Eq. (6.14) can be understood by noting that the elements of $[\mathbf{G}^T\mathbf{G}]$ are just the dot products of the columns, $\mathbf{c}^{(i)}$, of the matrix, $\mathbf{G}$. Many of these dot products are clearly zero (meaning that $[\mathbf{G}^T\mathbf{G}]_{ij} = \mathbf{c}^{(i)T}\mathbf{c}^{(j)} = 0$, when $i \neq j$). For instance, the second column, $\cos(\Delta wt)$, is symmetric about its midpoint, while the third column, $\cos(\Delta wt)$, is antisymmetric about it, so their dot product is necessarily zero (see Fig. 6.2B and C). What is not so clear—but nevertheless true—is that every column is orthogonal to every other. We will not derive this result here, for it requires rather nitty-gritty trigonometric manipulations (but see Note 6.2). Its consequence is that the least-squares solution, $\mathbf{m}^{est} = [\mathbf{G}^T\mathbf{G}]^{-1}\mathbf{G}^T\mathbf{d}^{obs}$ requires only matrix multiplication, and not matrix inversion. Furthermore, when the data are uncorrelated, then the model parameters, $\mathbf{m}^{est}$, which represent the amplitudes of the sines and cosines, are also uncorrelated, as $\mathbf{C}_m = \sigma_d^2[\mathbf{G}^T\mathbf{G}]^{-1}$ is diagonal. Furthermore, all but the first and last model parameters have equal variance.

In a script, the least-squares solution is computed as follows:

MATLAB

```
% edama_06_05: a.s.d. of the Neuse River hydrograph
. . .
% solve least squares problem using
% analytic inverse of GTG
gtgi = 2*ones(M,1)/N;
gtgi(1)=1/N;
gtgi(M)=1/N;
mest = gtgi .* (G'*d);
```

Python

```
# edapy_06_05: a.s.d. of the Neuse River hydrograph
. . .
# solve least squares problem using
# analytic formula for inv(GT*G)
GTd = np.matmul( G.T, d );
gtgi = 2*np.ones((M,1))/N;
gtgi[0,0]=1/N;
gtgi[M-1,0]=1/N;
mest = np.multiply( gtgi, GTd );
```

The column-vectors,

$$\left[A_1^2, A_2^2 + B_2^2, \cdots, A_{N/2}^2 + B_{N/2}^2, A_{N/2+1}^2\right]^T$$
and
$$\left[\sqrt{A_1^2}, \sqrt{A_2^2 + B_2^2}, \cdots, \sqrt{A_{N/2}^2 + B_{N/2}^2}, \sqrt{A_{N/2+1}^2}\right]^T \tag{6.15}$$

are called the *power spectral density (p.s.d.)* and *amplitude spectral density (a.s.d)* of the time series, respectively. Either can be used to quantify the overall intensity of oscillations at any given frequency, irrespective of the phase. In a script, the power spectral density, **s**, is computed from $\mathbf{m}^{est}$ as follows:

MATLAB

```
% eda06_05: a.s.d. of the Neuse River hydrograph
. . .
% compute power spectral density s
s=zeros(Nw,1);
s(:,1)=sqrt(mest(1)^2); % zero frequency
for i = [1:M/2-1] % interior points
    j = 2*i;
    k = j+1;
    s(i+1) = sqrt(mest(j)^2 + mest(k)^2);
end
s(Nw) = sqrt(mest(M)^2); % Nyquist frequency
```

Python

```
# edapy_06_05: a.s.d. of the Neuse River hydrograph
. . .
# compute power spectral density s
s=np.zeros((Nw,1));
s[0,0]=np.sqrt(mest[0,0]**2); # zero frequency
for i in range(1,Mo2): # interior points
    j = 2*i-1;
    k = j+1;
    s[i,0] = sqrt(mest[j,0]**2 + mest[k,0]**2);
s[Nw-1,0]= sqrt(mest[M-1,0]**2); # Nyquist frequency
```

When applied to the Neuse River discharge data, this method produces an amplitude spectral density that is largest at low frequencies (Fig. 6.4A). In such cases, a graph with

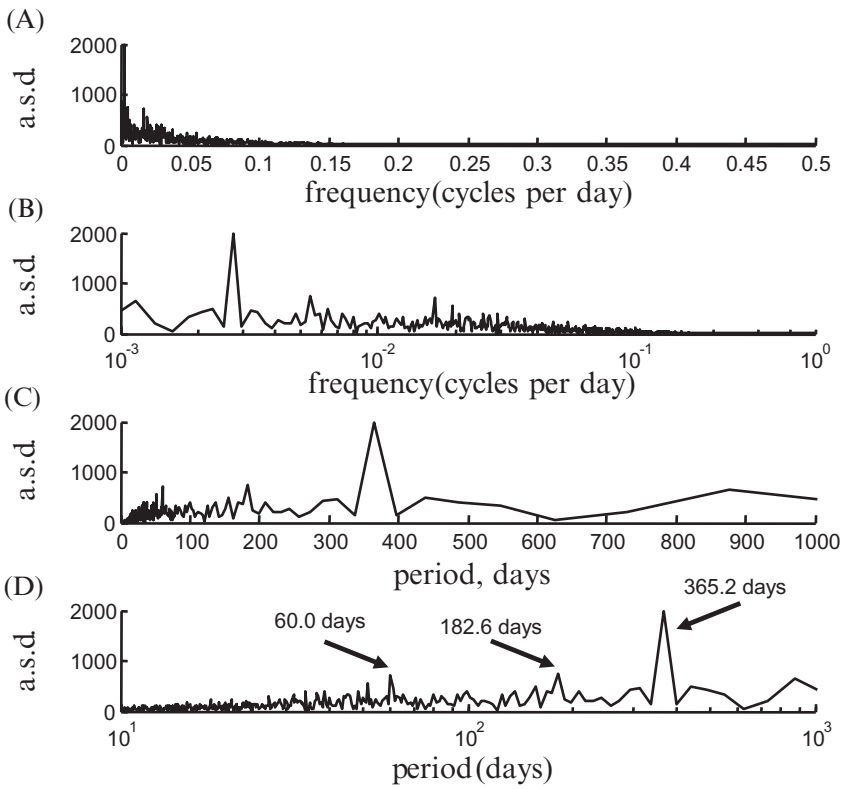


Fig. 6.4 Amplitude spectral density (a.s.d.) of the Neuse River discharge dataset. (A) Linear frequency axis. (B) Logarithmic frequency axis. (C) Linear period axis. (D) Logarithmic period axis. Scripts `edama_06_05` and `edapy_06_06`.

a logarithmic frequency axis (Fig. 6.4B) allows one to see details that are squeezed near the origin of a linear plot (Fig. 6.4A). In some applications, plotting amplitude spectral density as a function of period, as contrasted to frequency, may be preferable (as in this case, where the annual period of 365.2 days is more recognizable than the equivalent frequency of 0.002738 cycles/day). The amplitude spectral density consists of several peaks, superimposed on a "noisy background" that gradually declines with period. The three largest peaks have periods of 365.2, 182.6, and 60.2, (one, one-half, and one-sixth years) and are associated with oscillations in stream flow caused by seasonal fluctuations in rainfall. A common practice is to omit the zero-frequency element of the spectral density from the plots (as was done here). It only reflects the mean value of the time series and is not really relevant to the issue of periodicities. Large values can require a vertical scaling that obscures the rest of the plot. The scripts, above, worked fine calculating the spectral density of an $N = 4000$ length time series. However, the $N \times N$ matrix, $\mathbf{G}$, becomes prohibitively large for larger datasets. Fortunately, a very efficient algorithm, called the *fast Fourier transform (FFT)*, has been discovered for solving for $\mathbf{m}^{est}$ that requires much less storage and much less computation than "brute force" multiplication by $\mathbf{G}^T$. We will return to this issue later in the chapter.

6.3 Going complex

Many of the formulas of the previous section can be substantially simplified by switching from sines and cosines to complex exponentials, utilizing *Euler's formulas*:

$$\exp(i\omega t) = \cos(\omega t) + i \sin(\omega t) \quad \text{and} \quad \exp(-i\omega t) = \cos(\omega t) - i \sin(\omega t) \tag{6.16}$$

Here, i is an imaginary unit. These formulas imply

$$\cos(\omega t) = \frac{\exp(i\omega t) + \exp(-i\omega t)}{2} \quad \text{and} \quad \sin(\omega t) = \frac{\exp(i\omega t) - \exp(-i\omega t)}{2i} \tag{6.17}$$

The main complication (besides the need to use complex numbers) is that both positive and negative frequencies are needed in the Fourier series. Previously, we paired sines and cosines; now we will pair complex exponentials with positive and negative frequencies:

$$d(t) = \cdots A_n \cos(\omega_n t) + B_n \sin(\omega_n t) \cdots = \cdots C_n^- \exp(-i\omega_n t) + C_n^+ \exp(i\omega_n t) \cdots \tag{6.18}$$

Here, C_n^- and C_n^+ are the coefficients of the negative-frequency and positive-frequency terms, respectively. The requirement that these two different representations be equal implies a relationship between the As and Bs and the Cs. We write out the Cs in

terms of their real and imaginary parts, $C^- = C_R^- + iC_I^-$ and $C^+ = C_R^+ + iC_I^+$ and perform the multiplication explicitly:

$$\begin{aligned} &C^- \exp(-i\omega t) + C^+ \exp(i\omega t) = \\ &= \left(C_R^- + iC_I^-\right)\{\cos(\omega t) - i\sin(\omega t)\} + \left(C_R^+ + iC_I^+\right)\{\cos(\omega t) + i\sin(\omega t)\} \\ &= \left(C_R^- + C_R^+\right)\cos(\omega t) + \left(C_I^- - C_I^+\right)\sin(\omega t) + i\left(C_I^- + C_I^+\right)\cos(\omega t) + i\left(-C_R^- + C_R^+\right)\sin(\omega t) \end{aligned} \tag{6.19}$$

As the time series, $d(t)$, is real, the two imaginary terms must be zero. This happens when $C_I^- = -C_I^+$ and $C_R^- = C_R^+$; that is, when C^- and C^+ are complex conjugates of each other, so that $C^- = C_R - iC_I$ and $C^+ = C_R + iC_I$. Comparing Eqs. (6.18), (6.19), we find

$$A = \left(C_R^- + C_R^+\right) = 2C_R \quad \text{and} \quad B = \left(C_I^- - C_I^+\right) = -2C_I \tag{6.20}$$

The power spectral density is now computed from $C_R^2 + C_I^2$, which is equivalent to C times is complex conjugate; that is $C_R^2 + C_I^2 = C^* C = |C|^2$. Here, the asterisk means complex conjugation.

We can represent a real function as a Fourier series that involves sines and cosines of real amplitudes, A and B, respectively, or alternatively, as a Fourier series that involves complex exponentials with complex coefficients, C:

$$\begin{aligned} &d_k = \sum_{n=1}^{\frac{N}{2}+1}\{A_n \cos(\omega_n t) + B_n \sin(\omega_n t)\} \\ &\text{with } \left(\omega_1, \omega_2, \omega_3, \cdots, \omega_{N/2+1}\right) = (0, \Delta\omega, 2\Delta\omega, \cdots \tfrac{1}{2}N\Delta\omega) \end{aligned} \tag{6.21a}$$

$$\begin{aligned} &d_k = \frac{1}{N}\sum_{n=1}^{N} C_n \exp(i\omega_n t) \\ &\text{with } \left(\omega_1, \omega_2, \omega_3, \cdots, \omega_{N/2+1}, \omega_{N/2+2} \cdots \omega_N\right) = \\ &(0, \Delta\omega, 2\Delta\omega, \cdots \tfrac{1}{2}N\Delta\omega, -(\tfrac{1}{2}N - 1), \cdots, -2\Delta\omega, -\Delta\omega) \end{aligned} \tag{6.21b}$$

Note the nonintuitive ordering of the frequencies in the summation of the complex exponentials, which we will discuss in detail below. The factor of N^{-1} has been added to the complex summation in order to match the convention used in the literature, so that now $(2/N)C_n = A_n - iB_n$.

The solution of Eq. (6.21b) for the coefficients, C_n, requires the complex version of least squares (see Note 4.1). We will not discuss it in any detail here, except to note that the matrix, $[\mathbf{G}^T * \mathbf{G}]^{-1} = N^{-1}\mathbf{I}$, is diagonal (see Note 6.2) so that, like the sine and cosine version of the D.F.T., matrix inversion is not required. The complex coefficients, C_j are calculated from the time series, $d(t_n)$, by

$$C_j = \sum_{k=1}^{N} d(t_n) \exp\left(-i\omega_j t_n\right) \tag{6.22}$$

In the sine and cosine case, the sum has $N/2+1$ pairs of sines and cosines, but the coefficients of the sines in the first and last pairs are zero, so the total number of unknowns is $2(N/2+1)-2=N$. In the complex exponential case, the sum has N complex coefficients, but except for the first and last, they occur in complex conjugate pairs, so only the $N/2+1$ coefficients of the nonnegative frequencies count for the unknowns. Each of these has a real and imaginary part, except for the first and the last, which are purely real, so the number of unknowns is once again $2(N/2+1)-2=N$. Note that if the data were complex, the coefficients would all be different; that is, N complex coefficients are needed to represent N complex data.

We return now to the issue of the ordering of the frequencies, which is to say, the order of the model parameters, $\mathbf{m}$. The ordering presented in Eq. (6.21b) has the nonnegative frequencies first and the negative frequencies last. This ordering, while nonintuitive, is actually more useful than a strictly ascending ordering, because the nonnegative frequencies are really the only ones needed when dealing with real time series. The negative frequencies, being complex conjugates of the positive ones, are redundant. However, the ordering has a more subtle rationale related to aliasing. The negative frequencies correspond exactly to the positive frequencies that one would get, if the positive ordering were extended past the Nyquist frequency. For example, the last frequency, $-\Delta\omega$, is exactly equivalent to $+(N-1)\Delta\omega$, in the sense that both correspond to the same complex exponential.

In scripts, the arrays of frequencies are created as follows:

MATLAB

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
% generic frequency setup
fmax=1/(2.0*Dt); % nyquist frequency
df=fmax/(N/2);   % frequency sampling
Nf=N/2+1;        % number of non-negative frequencies
% f is full frequency vector:
% f = [0, Df, 2Df ... fmax, -fmax+Df, ... -Df]
f=df*[0:N/2,-N/2+1:-1]';
dw=2*pi*df; % angular frequency sampling
w=2*pi*f;   % full angular frequency vector
Nw=Nf;      % number of non-negative angular frequencies
fpos = df*[0:N/2]';  % non-negative frequencies
wpos = 2*pi*fpos;    % non-negative angular frequencies
```

Python

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
# generic frequency setup
fmax=1/(2.0*Dt); # nyquist frequency
Df=fmax/No2;     # frequency sampling
Nf=No2+1;        # number of non-negative frequencies
# f is full frequency vector:
# f = [0, Df, 2Df ... fmax, -fmax+Df, ... -Df]
f=eda_cvec(
     Df*np.concatenate(
     (np.linspace(0,No2,Nf),np.linspace(-No2+1,-1,No2-1)),
     axis=0 ));
Dw=2*pi*Df;        # angular frequency sampling
w=2*pi*f;          # full angular frequency vector
Nw=Nf;             # number of non-negative angular f's
fpos = eda_cvec(Df * np.linspace(0,No2,Nf)); # non-neg f's
wpos=2*pi*fpos;    # non-negative angular frequencies
```

Here, `N` is the length of the data, again presumed to be an even integer. The quantities, `Nf` and `Nw`, are the numbers of nonnegative frequencies. The *Fast Fourier Transform (FFT)* algorithm solves for the complex Fourier coefficients very efficiently, and should always be used in preference to the summation in Eq. (6.22). In scripts, it is invoked as:

MATLAB

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
% Compute fourier coefficients
mest = fft(d);
```

Python

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
# Compute fourier coefficients. Be careful not to forget
# the axis=0 (fft over elements of column). The default
# axis=1 (fft over elements of rows) won't work!
mest = eda_cvec( np.fft.fft(d, axis=0) );
```

The amplitude spectral density is computed as follows:

MATLAB

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
% compute amplitude spectral density
s=abs(mest(1:Nw)); % a.s.d.
```

Python

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
# amplitude spectrum (note is un-normalized)
mestpos = eda_cvec( mest[0:Nf,0] ); # pos f's only
s = np.abs(mestpos);                # a.s.d.
```

Note that the amplitude spectral density is computed from the complex absolute value of the coefficients; that is $|C_n| = |m_n^{est}|$. The results are, of course, identical to least-squares, but they are computed with orders of magnitude less time and storage requirements. The time series can be rebuilt from its Fourier coefficients by using the *Inverse Fast Fourier Transform (FFT)* algorithm much more quickly than by performing the summation in Eq. (6.21b):

MATLAB

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
% Inverse FFT
dnew = ifft(mest);
```

Python

```
# edapy_06_06: a.s.d. of the Neuse River hydrograph
. . .
# Inverse FFT: Don't forget the axis=0!
dnew = eda_cvec( np.real(np.fft.ifft(mest,axis=0)));
```

6.4 Lessons learned from the integral transform

The Discrete Fourier Transform and its inverse are very closely related to integrals called the *Fourier transform* and the *Inverse Fourier Transform*, as can be seen by a side-by-side comparison of the two:

$$\text{Fourier transform} \qquad\qquad \text{D.F.T.} \tag{6.23a}$$

$$\widetilde{d}(\omega) = \int_{-\infty}^{+\infty} d(t)\exp\left(-i\omega t\right)\mathrm{d}t \quad \text{and} \quad C_j = \sum_{k=1}^{N} d_k \exp\left(-i\omega_j t_k\right) \tag{6.23b}$$

$$\text{Inverse Fourier transform} \qquad\qquad \text{Inverse D.F.T.} \tag{6.23c}$$

$$d(t) = \frac{1}{2\pi}\int_{-\infty}^{+\infty} \widetilde{d}(\omega)\exp\left(+\,i\omega t\right)\mathrm{d}\omega \quad \text{and} \quad d_k = \frac{1}{N}\sum_{n=1}^{N} C_n \exp\left(+\,i\omega_n t_k\right) \tag{6.23d}$$

The Fourier Transform Integral converts the function, $d(t)$, into the its *Fourier transform*, the function, $\widetilde{d}(\omega)$. Similarly, the D.F.T. summation converts a time series, d_k, into its D.F.T., the frequency-series, C_j. If we assume that the $d(t)$ is nonzero only between 0 and t_{max}, then the integral can be approximated by its Riemann sum:

$$\widetilde{d}\left(\omega_j\right) = \int_0^{t_{max}} d(t)\exp\left(-i\omega_j t\right)\mathrm{d}t \approx \Delta t \sum_{k=1}^{N} d(t_k)\exp\left(-i\omega_j t_k\right) \tag{6.24}$$

Comparing with Eq. (6.23b), we deduce that $\widetilde{d}\left(\omega_j\right) \approx \Delta t C_j$, that is, the Fourier Transform and D.F.T. coefficients differ only by the constant, Δt. A similar relationship holds between the inverse transform and the Fourier summation as well:

$$d\left(t_j\right) = \frac{1}{2\pi}\int_{-\infty}^{+\infty} \widetilde{d}(\omega)\exp\left(+\,i\omega t_j\right)\mathrm{d}\omega \approx \frac{N\Delta\omega}{2\pi}\,\frac{1}{N}\sum_{k=1}^{N} \widetilde{d}(\omega_k)\exp\left(+\,i\omega_k t_j\right) \tag{6.25}$$

Comparing with Eq. (6.23d), we deduce that $C_k = N\Delta\omega\widetilde{d}(\omega_k)/2\pi$. After applying the rule, $\Delta\omega\Delta t = 2\pi/N$, we again find $\widetilde{d}(\omega_k) \approx \Delta t C_k$.

In the context of this book, Fourier transforms are of interest because they can be more readily manipulated than D.F.T.s. Many of the relationships that are true for Fourier transforms will also be true—or approximately true—for D.F.T.s. We summarize a few of the most useful relationships below.

6.5 Normal curve

The Fourier transform of the Normal function, $d(t) = \exp(-a^2t^2)$, is

$$\begin{aligned}\widetilde{d}(\omega) &= \int_{-\infty}^{+\infty} \exp\left(-a^2t^2\right) \exp\left(-i\omega t\right) \mathrm{d}t \\ &= 2\int_{0}^{+\infty} \exp\left(-a^2t^2\right) \cos\left(\omega t\right) \mathrm{d}t = \frac{\sqrt{\pi}}{a} \exp\left(-\frac{\omega^2}{4a^2}\right)\end{aligned} \tag{6.26}$$

Here, we have expanded $\exp(-i\omega t)$ into $\cos(\omega t) - i\sin(\omega t)$. The integrand, $\sin(\omega t)\exp(-a^2t^2)$, consists of an odd function (the sine) of time multiplied by an even function (the exponential) of time. It is, therefore, odd and so its integral over $(-\infty, +\infty)$ is zero. The product, $\cos(\omega t)\exp(-a^2t^2)$, is an even function, so its integral on $(-\infty, +\infty)$ is twice its integral on $(0, +\infty)$. A standard table of integrals (e.g., integral 679 of the CRC Standard Mathematical Tables, Zwillinger, 1996) is used to evaluate the final integral.

If we write $a^2 = \frac{1}{2}\sigma^{-2}$, then the exponential has the form of a standard Normal curve with zero mean and variance, σ^2:

$$d(t) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{t^2}{2\sigma^2}\right) \quad \text{and} \quad \widetilde{d}(\omega) = \exp\left(-\frac{\omega^2}{2\sigma^{-2}}\right) \tag{6.27}$$

Thus, up to an overall normalization, the Fourier transform of a Normal curve with variance, σ^2, is a Normal curve with variance, σ^{-2}.

This result is extremely important because it quantifies how the widths of functions and the widths of their Fourier transforms are related. When $d(t)$ is a wide function, $\widetilde{d}(\omega)$ is narrow, and vice-versa. A spiky function, such as a narrow Normal curve, has a transform that is rich in high frequencies. A very smooth function, such as a wide Normal curve, has a transform that lacks high frequencies (Fig. 6.5).

6.6 Spikes

The relationship between the width of a function and its Fourier transform can be pursued further by defining the *Dirac Impulse Function*—a Normal curve in the limit of vanishing small variance:

$$\delta(t - t_0) = \lim_{\sigma \to 0} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t - t_0)^2}{2\sigma^2}\right) \tag{6.28}$$

This *generalized function* is zero everywhere, except at the point, t_0, where it is singular. Nevertheless, it has unit area. When the product of $\delta(t - t_0)$ and another function $f(t)$ is integrated, the result is just $f(t)$ evaluated at the point, $t = t_0$:

$$\int_{-\infty}^{+\infty} \delta(t - t_0) f(t) \, \mathrm{d}t = f(t_0) \tag{6.29}$$

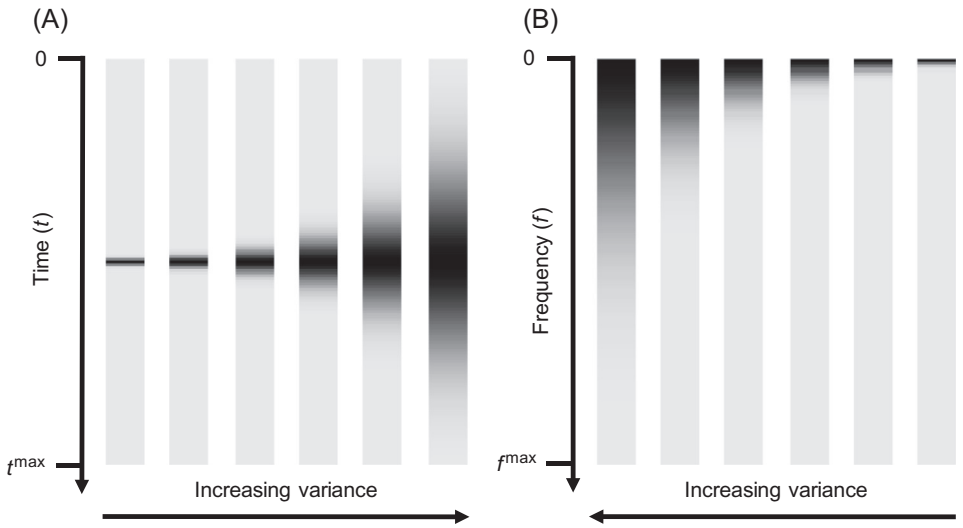


Fig. 6.5 (A) Shaded column-vectors of a series of Normal functions with increasing variance. (B) Corresponding amplitude spectral density. Script `edama_06_07` and `edapy_06_07`.

This result can be understood by noting that the Dirac function is nonzero only in a vanishingly small interval of time, t_0. Within this interval, the function, $f(t)$, is just the constant, $f(t_0)$. No error is introduced by replacing $f(t)$ with $f(t_0)$ everywhere and taking it outside the integral, which then integrates to unity.

The Fourier transform of a spike at $t_0 = 0$ is unity (Fig. 6.6):

$$\widetilde{d}(\omega) = \int_{-\infty}^{+\infty} \delta(t) \exp(-i\omega t)\, \mathrm{d}t = \exp(-i\omega t)\big|_{t=0} = 1 \tag{6.30}$$

This is the limiting case of an indefinitely narrow Normal function (see Eq. 6.28). The Dirac function, being "infinitely spiky," has a transform that contains all frequencies in equal proportions. The transform with a spike at $t = t_0$, is

$$\widetilde{d}(\omega) = \int_{-\infty}^{+\infty} \delta(t - t_0) \exp(-i\omega t)\, \mathrm{d}t = \exp(-i\omega t)\big|_{t=t_0} = \exp(-i\omega t_0) \tag{6.31}$$

Although it is an oscillatory function of time, its power spectral density is constant: $s(\omega) = \widetilde{d}^{*}(\omega)\widetilde{d}(\omega) = \exp(+i\omega t_0)\exp(-i\omega t_0) = 1$.

The Dirac function can appear in a function's Fourier transform as well. The transform of $\cos(\omega_0 t)$ is

$$\widetilde{d}(\omega) = \pi\{\delta(\omega - \omega_0) + \delta(\omega + \omega_0)\} \tag{6.32}$$

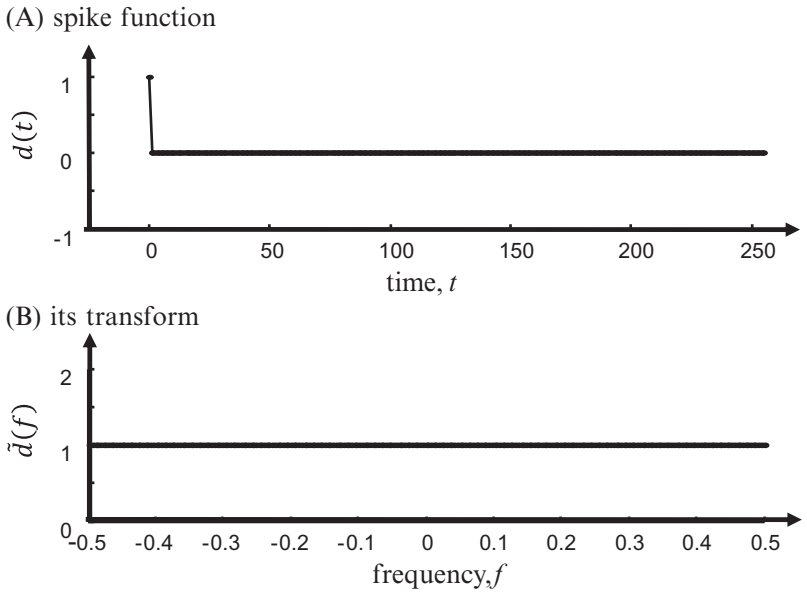


Fig. 6.6 (A) Spike function is zero except at time, $t = 0$. (B) Corresponding Fourier transform is unity. Scripts `edama_06_08` and `edapy_06_08`.

This formula can be verified by inserting it into the inverse transform:

$$\begin{aligned} d(t) &= \frac{1}{2\pi}\int_{-\infty}^{+\infty} \pi\{\delta(\omega-\omega_0)+\delta(\omega+\omega_0)\}\exp(i\omega t)\,\mathrm{d}\omega \\ &= \frac{\exp(i\omega_0 t)+\exp(-i\omega_0 t)}{2} = \cos(\omega_0 t) \end{aligned} \tag{6.33}$$

Here, we have employed Eq. (6.17). As one might expect, the transform of the pure sinusoid, $\cos(\omega_0 t)$, contains only two frequencies, $\pm\omega_0$.

6.7 Area under a function

The area, A, under a function, $d(t)$, is its Fourier transform evaluated at zero frequency; that is, $A = \widetilde{d}(\omega = 0)$:

$$\widetilde{d}(\omega=0) = \int_{-\infty}^{+\infty} d(t)\exp(0)\,\mathrm{d}t = \int_{-\infty}^{+\infty} d(t)\mathrm{d}t = A \tag{6.34}$$

as $\exp(0) = 1$ (Fig. 6.7). In a script, one way—though perhaps not the fastest—to compute area is

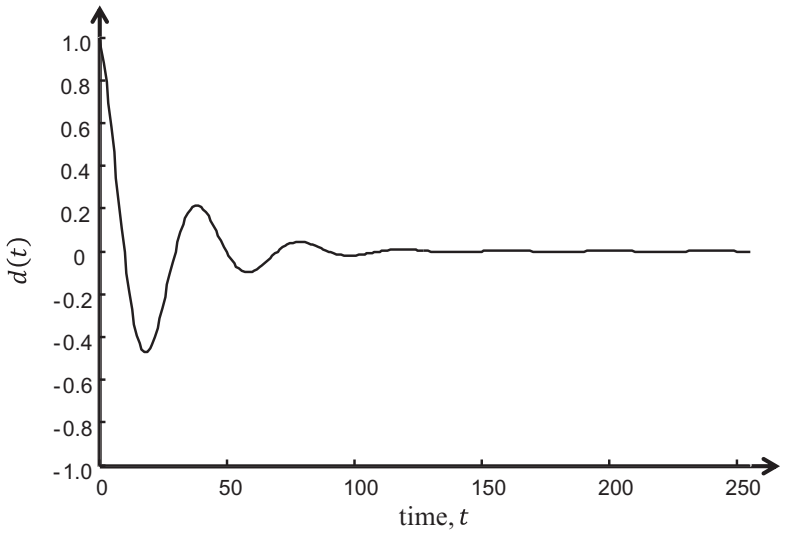


Fig. 6.7 The area under the exemplary function, $d(t)$, is computed by approximating the integral with a summation and by the Fourier method described in the text. Both give the same value, 2.0014. Scripts `edama_06_09` and `edapy_06_09`.

MATLAB

```
% edama_06_09: area under a curve using FFT
. . .
% fourier transform; note normalization factor of Dt
dt=Dt*fft(d);
area2 = real(dt(1));
```

Python

```
# edapy_06_09: area under curve using FFT
. . .
# Fourier transform; note normalization factor of Dt
# and use of axis=0 option.
dt = eda_cvec( Dt*np.fft.fft(d, axis=0) );
# area by fft
area2 = np.real(dt[0,0]);
```

In the scripts above, the $\omega = 0$ element of the Fourier transform, `dt`, is purely real, in the sense that its imaginary part is zero. Nevertheless, it is a complex variable that must to be converted to the real variable, `area2`, representing area.

6.8 Time-delayed function

Multiplying the transform, $\widetilde{d}(\omega)$ by the factor, $\exp(i\omega t_0)$, time-delays the function by an interval, t_0:

$$\begin{aligned}\widetilde{d}_{delayed}(\omega) &= \int_{-\infty}^{+\infty} d(t-t_0)\exp(-i\omega t)\,\mathrm{d}t = \int_{-\infty}^{+\infty} d(t')\exp\{-i\omega(t'+t_0)\}\,\mathrm{d}t' \\ &= \exp(-i\omega t_0)\int_{-\infty}^{+\infty} d(t')\exp\{-i\omega t'\}\,\mathrm{d}t' = \exp(-i\omega t_0)\,\widetilde{d}(\omega)\end{aligned} \tag{6.35}$$

Here, we use the transformation of variables, $t' = t - t_0$, noting that $\mathrm{d}t' = \mathrm{d}t$ and that $t' \to \pm\infty$ as $t \to \pm\infty$. In the literature, the process of modifying a Fourier transform by multiplication with a factor, $\exp(-i\omega t_0)$, is sometimes referred to as introducing a *phase ramp*, as it changes the phase by an amount proportional to frequency (i.e., by a ramp-shaped function):

$$\varphi(\omega) = \omega t_0 \tag{6.36}$$

The time-shift result appeared previously, when we were calculating the transform of a time-delayed spike (Eq. 6.31). The transform of the time-shifted spike differed from the transform of a spike at time zero by a factor of $\exp(-i\omega t_0)$. In scripts, the time delay is accomplished as follows (Fig. 6.8):

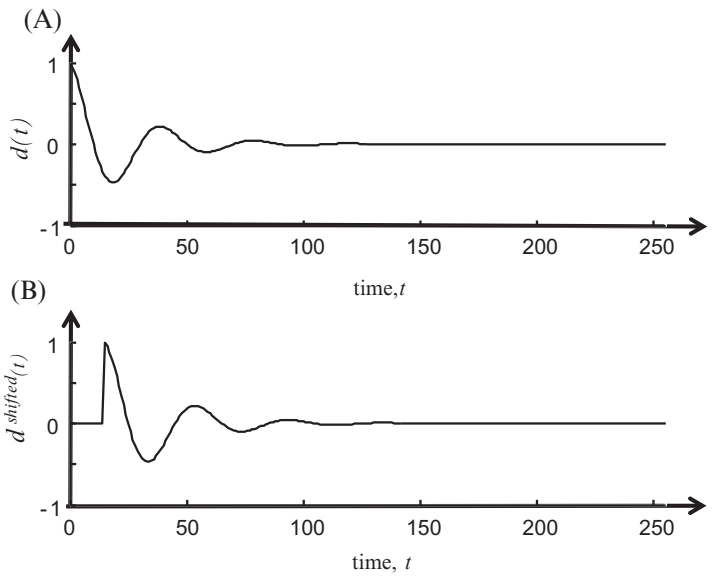


Fig. 6.8 The exemplary function, $d(t)$, is time shifted by an interval, $t_0 = 15$ using the Fourier method described in the text. Scripts `edama_06_10` and `edapy_06_10`.

MATLAB

```
% edama_06_10: time shift using FFT
. . .
% time shift
t0 = t(16);
ds=ifft(exp(-complex(0,1)*w*t0).*fft(d));
```

Python

```
# edapy_06_10: time shift using FFT
. . .
# time shift
t0 = t[15,0];
dt = (np.multiply(
    np.exp((complex(0,-1)*w*t0)), np.fft.fft(d, axis=0)));
```

Here, we use of the `complex(0,1)` function, and not the variable, `i`, to represent the imaginary unit. This is an error-avoidance strategy. Although we could initialize `i` to the imaginary unit, subsequent use of it, say as a for-loop variable, will overwrite this value and lead to incorrect results. See Note 1.1.

Nothing in the derivation requires that the time shift, t_0, be an integral multiple of the sampling interval, Δt. Consequently, this procedure can be used to time-shift a timeseries by a fractional number of samples. It can be used, for instance, to bring two time series with different start times into alignment.

6.9 Derivative of a function

Multiplying the transform, $\widetilde{d}(\omega)$ by the factor, $i\omega$, computes the transform of the derivative, $\mathrm{d}d/\mathrm{d}t$:

$$\begin{aligned}\int_{-\infty}^{+\infty} \frac{\mathrm{d}d}{\mathrm{d}t} \exp(-i\omega t)\,\mathrm{d}t &= d(t)\exp(-i\omega t)\big|_{-\infty}^{+\infty} - (-i\omega)\int_{-\infty}^{+\infty} d(t)\exp(-i\omega t)\mathrm{d}t \\ &= 0 + (i\omega)\widetilde{d}(\omega) = i\omega\widetilde{d}(\omega)\end{aligned} \tag{6.37}$$

Here, we have used integrations by parts, $\int u\,\mathrm{d}v = uv - \int v\,\mathrm{d}u$, together with the limit, $\exp(\pm i\omega t) \to 0$ as $t \to \pm\infty$. In scripts, the derivative can be computed as follows (Fig. 6.9):

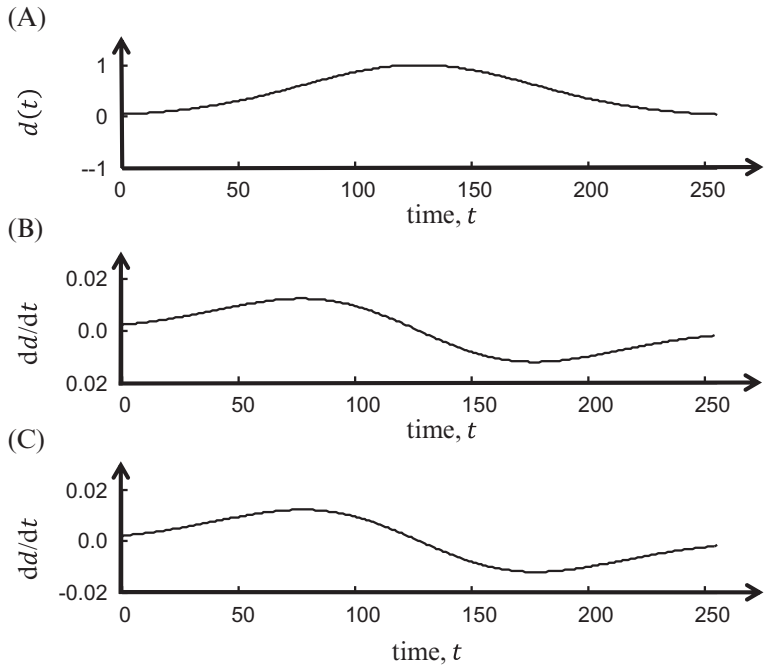


Fig. 6.9 (A) Exemplary function, $d(t)$. (B) Derivative of $d(t)$ as computed by finite differences. (C) Derivative as computed by the Fourier method described in the text. Scripts `edama_06_11` and `edapy_06_11`.

MATLAB

```
% edama_06_11: derivative using FFT
. . .
% derivative using fft
dddt=ifft(complex(0,1)*w.*fft(d));
```

Python

```
# eda06_11: derivative using FFT
. . .
# take Fourier transform and multiply by iw
# except for zero frequency
dt = np.multiply(complex(0,1)*w , np.fft.fft(d, axis=0));
# inverse Fourier transform
dddt = np.real((np.fft.ifft(dt,axis=0)));
```

6.10 Integral of a function

Dividing the transform, $\widetilde{d}(\omega)$ by the factor, $i\omega$, computes the transform of the indefinite integral, $A(t) = \int^t d(t')\ \mathrm{d}t'$:

$$\begin{aligned}\widetilde{A}(\omega) &= \int_{-\infty}^{+\infty}\int_{-\infty}^{t} d(t')\mathrm{d}t' \exp(-i\omega t)\,\mathrm{d}t \\ &= \int_{-\infty}^{t} d(t')\mathrm{d}t' \frac{\exp(-i\omega t)}{-i\omega}\Bigg]_{-\infty}^{+\infty} - \frac{1}{-i\omega}\int_{-\infty}^{+\infty} d(t)\exp(-i\omega t)\mathrm{d}t \\ &= 0 + \frac{1}{i\omega}\widetilde{d}(\omega) = \frac{1}{i\omega}\widetilde{d}(\omega)\end{aligned} \tag{6.38}$$

Here, we have used integrations by parts, $\int u\ \mathrm{d}v = uv - \int v\ \mathrm{d}u$, the fundamental theorem of calculus, $d(t) = (\mathrm{d}/\mathrm{d}t)\int_{-\infty}^{t} d(t')\ \mathrm{d}t'$ and the limit, $\exp(\pm i\omega t) \to 0$ as $t \to \pm\infty$. Note, however, that the zero-frequency value is undefined (as $1/0$ is undefined). As shown in Eq. (6.34), this value is the total area under the curve, so it functions as integration constant and must be set to an appropriate value. In a script, with the zero-frequency value set to zero, the integral is calculated as follows (Fig. 6.10):

MATLAB

```
% edama_06_12: integral using FFT
. . .
% take Fourier transform and divide by iw
% except set zero frequency to zero
wr=1.0./(complex(0,1)*w(2:N));
integral=ifft(fft(d).*[0,wr']');
```

Python

```
# edapy_06_12: integral using FFT
. . .
# take Fourier transform and divide by iw
# except set zero frequency to zero
wr=np.concatenate(
   (np.zeros((1,1)),np.reciprocal(complex(0,1)*w[1:N,0:1])) );
dt = np.multiply(wr, np.fft.fft(d, axis=0));
```

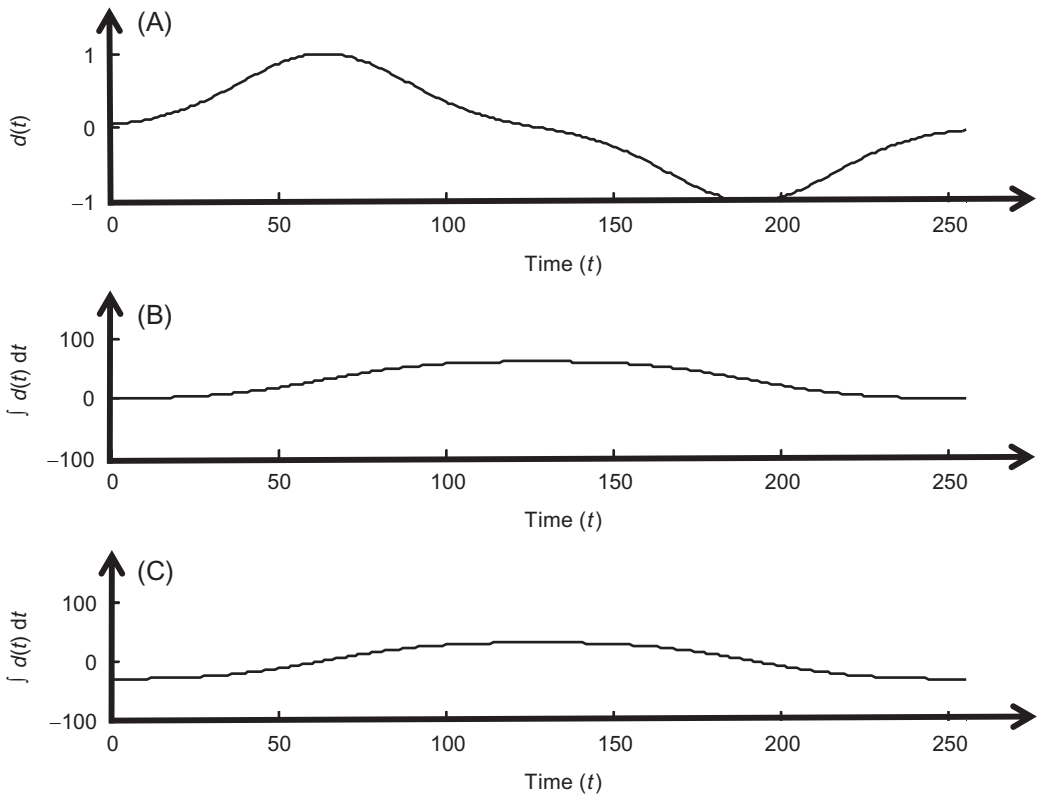


Fig. 6.10 (A) Exemplary function, $d(t)$. (B) Integral of $d(t)$ as computed by Riemann summation. (C) Integral as computed by the Fourier method described in the text. Notice that the two integrals differ by a constant offset. Scripts `edama_06_12` and `edapy_06_12`.

6.11 Convolution

Finally, we derive a result that pertains to the *convolution operation*, which will be developed further and utilized heavily in subsequent chapters. Given two functions, $f(t)$ and $g(t)$, their *convolution* is another function of time, $h(t)$, written symbolically as $h = f^* g$ and defined as:

$$h(t) = f(t)^* g(t) = \int_{-\infty}^{+\infty} f(\tau)\, g(t-\tau)\, \mathrm{d}\tau \tag{6.39}$$

Here, the asterisk (*) mean "convolution," not multiplication. The transform of the convolution is:

$$\begin{aligned}\widetilde{h}(\omega) &= \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} f(\tau)g(t-\tau)\mathrm{d}\tau \exp(-i\omega t)\,\mathrm{d}t \\ &= \int_{-\infty}^{+\infty} f(\tau)\int_{-\infty}^{+\infty} g(t-\tau)\exp(-i\omega t)\,\mathrm{d}t\,\mathrm{d}\tau \\ &= \int_{-\infty}^{+\infty} f(\tau)\int_{-\infty}^{+\infty} g(t')\exp\{-i\omega(t'+\tau)\}\,\mathrm{d}t'\,\mathrm{d}\tau \\ &= \int_{-\infty}^{+\infty} f(\tau)\exp(-i\omega\tau)\,\mathrm{d}\tau\int_{-\infty}^{+\infty} g(t')\exp(-i\omega t')\,\mathrm{d}t' \\ &= \widetilde{f}(\omega)\widetilde{g}(\omega)\end{aligned} \tag{6.40}$$

Here, we have used the transformation of variables, $t' = t - \tau$. Thus, the transform of the convolution of two functions is the product of their transforms.

6.12 Nontransient signals

Previously, in developing the relationship between the Fourier Transform and the D.F.T., we assumed that the function of interest, $d(t)$, was zero outside of the time window of observation. Only *transient signals* have this property; we can theoretically record the whole phenomenon, as it lasts only for a finite time. An equally common scenario is one in which the data represent just a portion of an indefinitely long phenomenon that has no well-defined beginning or end. Both the Neuse River Hydrograph and Black Rock Forest temperature datasets are of this type.

Many nontransient signals do not vary dramatically in overall pattern from one time window of observation to another (meaning that their statistical properties are *stationary*; that is, constant with time). One parameter that is independent of the window length is the power, P:

$$P = \frac{1}{T}\int_0^T [d(t)]^2\,\mathrm{d}t \approx \frac{\Delta t}{N\Delta t}\sum_{n=1}^{N} d_n^2 = \frac{1}{N}\mathbf{d}^T\mathbf{d} \tag{6.41}$$

In some cases, such as when $d(t)$ represents velocity, P literally is power, that is energy per unit time. Usually, however, the word is understood in the more abstract sense to mean the overall size of a signal that is fluctuating about zero. Note that when the data have zero mean, $P = N^{-1}\mathbf{d}^T\mathbf{d}$ is the estimated variance of $\mathbf{d}$. In this special case, the power is equivalent to the variance of the signal, $d(t)$.

The power in a time series has a close relationship to its power spectral density. A time series is related to its Fourier transform by the linear rule, $\mathbf{d}=\mathbf{Gm}$, where $\mathbf{m}$ is a column-vector of Fourier amplitudes. If we were to use the sines and cosines representation of the Fourier transform, then the matrix, $\mathbf{G}$, is given in Eq. (6.13). Substituting $\mathbf{d}=\mathbf{Gm}$ into Eq. (6.41) yields

$$P = \frac{1}{N}\mathbf{d}^T\mathbf{d} = \frac{1}{N}[\mathbf{Gm}]^T[\mathbf{Gm}] = \frac{1}{N}\mathbf{m}^T\left[\mathbf{G}^T\mathbf{G}\right]\mathbf{m} \tag{6.42}$$

According to Eq. (6.14), $\mathbf{G}^T\mathbf{G}=(N/2)\mathbf{I}$ (except for the first and last coefficient, which we shall ignore). Substituting this relationship yields:

$$P = \frac{1}{2}\frac{N}{2}\mathbf{m}^T\mathbf{m} = \frac{1}{2}\sum_{i=1}^{\frac{N}{2}+1}\left(A_i^2+B_i^2\right) = \frac{1}{2}\sum_{i=1}^{\frac{N}{2}+1}\frac{4}{N^2}[C_i]^2 = \frac{2}{(\Delta t)^2N^2\Delta f}\int_0^{f_{ny}}\left|\widetilde{d}(f)\right|^2 \mathrm{d}f \tag{6.43}$$

This result is called *Parseval's Theorem*. Here, the As and Bs are the coefficients in the cosines and sines representation of the Fourier series (Eq. 6.13) and the Cs are the coefficients of the DFT (Eq. 6.21b). As was shown in Section 6.3, the two representations are related by $(2/N)C_i=A_i-iB_i$. The Fourier transform is approximately, $\widetilde{d}(f)=\Delta t C_i$. If we *define* the power spectral density, $s^2(f)$, of a nontransient signal as

$$s^2(f) = \frac{2}{T}\left|\widetilde{d}(f)\right|^2 \quad \text{then} \quad P = \int_0^{f_{ny}} s^2(f)\,\mathrm{d}f \tag{6.44}$$

Here, we have used the relations $T=N\Delta t$ and $\Delta T\Delta f=N^{-1}$ to reduce $2/\{(\Delta t)^2N^2\Delta f\}$ to $2/T$. The power is the integral of the power spectral density from zero frequency to the Nyquist frequency. This equivalence is explored in scripts `edama_06_14` and `edapy_06_14`. In a script, the power spectral density is computed as follows:

MATLAB

```
% edama_06_15: p.s.d. of the Neuse River hydrograph
. . .
% compute power spectral sensitivity, s2
dbar=Dt*fft(d); % fourier transform
T = N*Dt; % duration of time series
s2 = (2/T) * abs(dbar(1:Nf)).^2; % p.s.d.
```

Python

```
# edapy_06_15: p.s.d. of the Neuse River hydrograph
. . .
# take Fourier transform and divide by iw
# except set zero frequency to zer
wr=np.concatenate(
  (np.zeros((1,1)),np.reciprocal(complex(0,1)*w[1:N,0:1])) );
dt = np.multiply(wr, np.fft.fft(d, axis=0));
```

Here, the data, `d`, are presumed to have sampling interval, `Dt`, and length, `N`.

This simple generalization of the idea of spectral density is closely connected to the limitation that we cannot take the Fourier transform of the whole phenomenon, for it is indefinitely long, but only a portion of it. Nevertheless, we would like for our results to be relatively insensitive to the length of the time window. The factor of $1/T$ in Eq. (6.44) normalizes for window length.

Suppose the units of $d(t)$ are u. Then, the Fourier transform, $\widetilde{d}(\omega)$, has units of u-s and the power spectral density has units of $u^2s^2/s = u^2s = u^2/\text{Hz}$. For example, for discharge data with units of cubic feet per second, $u = ft^3/s$, and power spectral density has units of ft^3, or ft^3/s per Hz (in the Neuse River hydrograph shown in Fig. 6.11, we use the mixed units of ft^3/s per cycle/day). The amplitude spectral density has units of the square root of the power spectral density; that is, u $\text{Hz}^{-½}$.

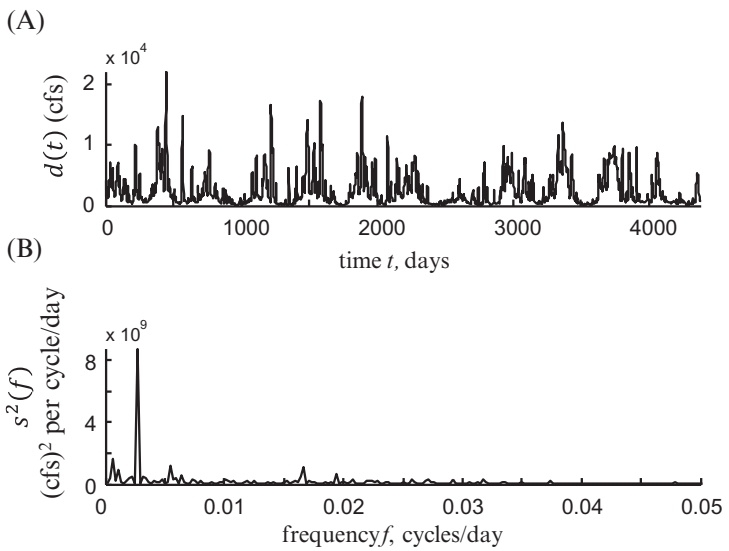


Fig. 6.11 (A) Neuse River hydrograph, $d(t)$. (B) Its power spectral density, $s^2(f)$. Scripts `edama_06_15` and `edapy_06_15`.

We have yet to discuss two important elements of working with spectra: First, we have made no mention of confidence limits, yet these are important in determining whether an observed periodicity (i.e., an observed spectral peak) is statistically significant. The reason we have omitted it so far is that the power spectral density is not a linear function of the model parameters, but instead depends on the sum of squares of the model parameters. We lack the tools to propagate error between the data and results in this case. Second, we have made no mention of the *tapering* process that is often used to prepare data before computing its Fourier transform. We will address these important issues in Chapters 11 and 8, respectively (Crib Sheets 6.1 and 6.2).

CRIB SHEET 6.1 Experimental design for spectra

Questions to ask yourself

What is the highest frequency f^{high} that you need to detect?
What is the minimum frequency separation δf of spectral peaks that you need to resolve?
How much data N can you record and analyze?

Important quantities

duration of recording, T
sampling interval, Δt
number of samples (amount of data), $N = T/\Delta t$
Nyquist (maximum) frequency, $f^{max} = 1/(2\Delta t)$
frequency spacing, $\Delta f = f^{max}/(N/2)$

Design principles

The Nyquist frequency must be greater than the highest frequency you need to detect, and preferably at least twice as high

$$f^{max} \approx 2 f^{high}$$

The frequency sampling must be smaller than the minimum frequency separation you need to resolve, and preferably ten times smaller

$$\Delta f \approx \delta f / 10$$

However, you need to verify that you have the ability to record and analyze the corresponding amount of data

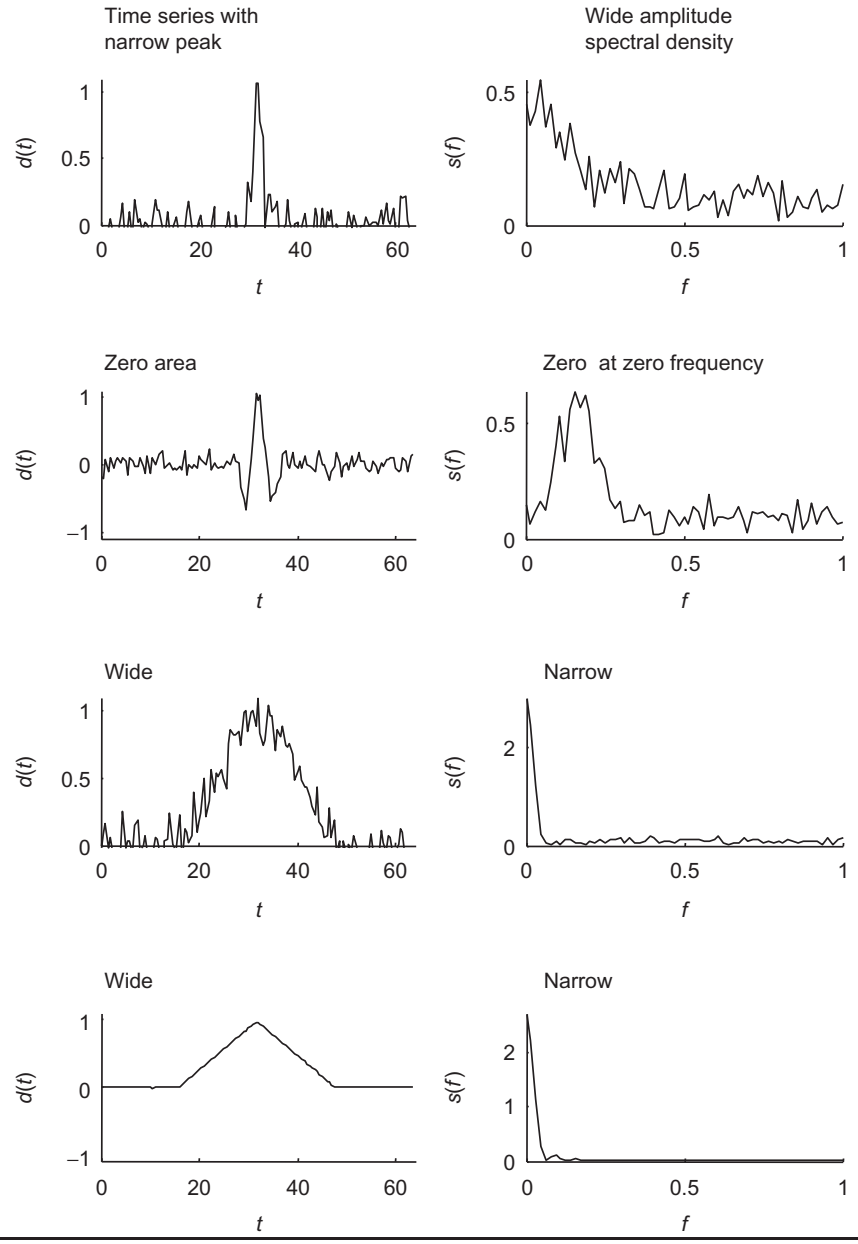
Design formulas

$$\Delta t = 1/(2 f^{max}) \approx 1/ f^{high}$$

$$N = 2 f^{max}/\Delta f \approx 40 f^{high}/\delta f$$

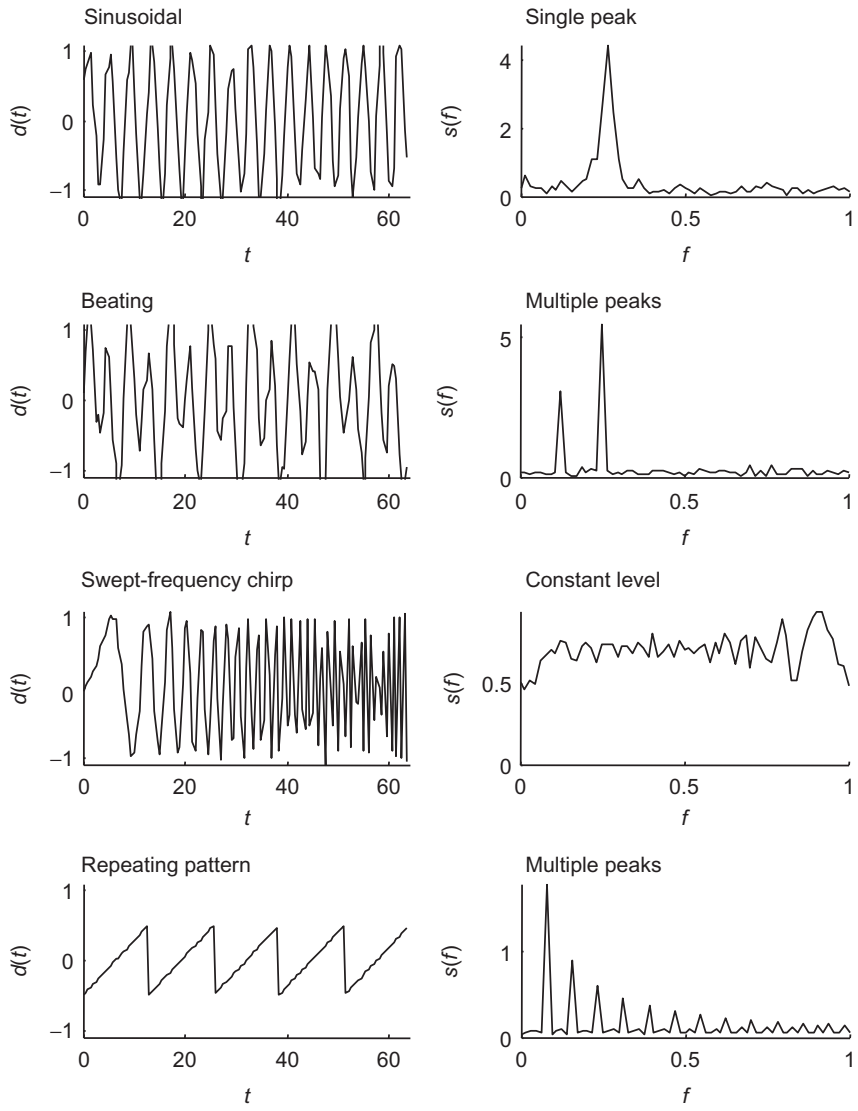
$$T = N\Delta t \approx 40/\delta f$$

CRIB SHEET 6.2 Exemplary pairs of time series *d*(*t*) and its amplitude spectral density *s*(*f*)



Continued

CRIB SHEET 6.2 Exemplary pairs of time series $d(t)$ and its amplitude spectral density $s(f)$—cont'd



scripts `edama_06_13` and `edapy_06_13`

Problems

6.1. Write a script that uses the fast Fourier transform to differentiate the Neuse River Hydrograph dataset. Plot the results.

6.2. What is the Fourier transform of $\sin(\omega_0 t)$? Compare it to the transform of $\cos(\omega_0 t)$.

6.3. The scripts, `edama_06_15` and `edapy_06_15` creates a file, Data/noise.txt, containing Normally distributed random time series, $d(t)$, with zero mean, unit variance and sampling interval $\Delta t = 1$. (A) Compute and plot the power spectral density of this time series. (B) Create a second time series, $a(t)$, that is a moving window average of $d(t)$; that is, each point in $a(t)$ is the average of, say L, neighboring points in $d(t)$. (C) Compute and plot the power spectral density of $a(t)$ for a suite of values of L. Comment on the results.

6.4. Suppose that you needed to compute the DFT of the function

$$d(t) = \exp\left(-\frac{t^2}{2\sigma^2}\right)$$

This function is centered on $t = 0$, and therefore has nonnegligible values for points to the left of the origin. Unfortunately, we have defined the time and data column-vectors, **t** and d, to start at time zero, so there seems to be no place to put these data values. One solution to this problem is to shift the function to the center of the time window, say by an amount, t_0, compute its Fourier transform, and then multiply the transform by a phase factor, $\exp(i\omega t_0)$, that shifts it back. Another solution relies on the fact that, in discrete transforms, both time and frequency suffer from aliasing. Just as the last frequencies in the transform were large positive frequencies and small negative frequencies, the last points in the time series are simultaneously

$$\mathbf{t} = \left[\cdots (N-3)\Delta t \;\; (N-2)\Delta t \;\; (N-1)\Delta t\right]^T$$

and

$$\mathbf{t} = \left[\cdots -3\Delta t \;\; -2\Delta t \;\; -\Delta t\right]^T$$

Therefore, one simply puts the negative part of $d(t)$ at the right-hand end of $d(t)$. Write a script to try both methods and check that they agree.

6.5. Compute and plot the amplitude spectral density of a cleaned version of the Black Rock Forest temperature dataset. (A) What are its units? (B) Interpret the periods of any spectral peaks that you find.

Further reading

Zwillinger, D., 1996. CRC Standard Mathematical Tables and Formulae, thirtieth ed. CRC Press.